DOM Manipulation and Click Events with JavaScript Review

Working with the DOM and Web APIs

- **API**: An API (Application Programming Interface) is a set of rules and protocols that allow software applications to communicate wit each other and exchange data efficiently.
- **Web API**: Web APIs are specifically designed for web applications. These types of APIs are often divided into two main categories: browser APIs and third-party APIs.
- **Browser APIs**: These APIs expose data from the browser. As a web developer, you can access and manipulate this data using JavaScr
- **Third-Party APIs**: These are not built into the browser by default. You have to retrieve their code in some way. Usually, they will have detailed documentation explaining how to use their services. An example is the Google Maps API, which you can use to display interactive maps on your website.
- **DOM**: The DOM stands for Document Object Model. It's a programming interface that lets you interact with HTML documents. Wit the DOM, you can add, modify, or delete elements on a webpage. The root of the DOM tree is the `html` element. It's the top-level container for all the content of an HTML document. All other nodes are descendants of this root node. Then, below the root node, w find other nodes in the hierarchy. A parent node is an element that contains other elements. A child node is an element that is contai within another element.
- `navigator` **Interface**: This provides information about the browser environment, such as the user agent string, the platform, and th version of the browser. A user agent string is a text string that identifies the browser and operating system being used.
- `window` **Interface**: This represents the browser window that contains the DOM document. It provides methods and properties for interacting with the browser window, such as resizing the window, opening new windows, and navigating to different URLs.

Working with the `querySelector()`, `querySelectorAll()` and `getElementById()` Methods

- `getElementById()` **Method**: This method is used to get an object that represents the HTML element with the specified `id`. Remember that ids must be unique in every HTML document, so this method will only return one Element object.

index.html index.js

```
1  <div id="container"></div>
2  <script src="./index.js"></script>
```

Open Sandbo

- `querySelector()` **Method**: This method is used to get the first element in the HTML document that matches the CSS selector pass as an argument.

index.html index.js

```
<section class="section"></section>
<script src="./index.js"></script>
```

Open Sandbo

- `querySelectorAll()` **Method**: You can use this method to get a list of all the DOM elements that match a specific CSS selector.

index.html index.js

```
<ul class="ingredients">
  <li>Sugar</li>
  <li>Milk</li>
  <li>Eggs</li>
</ul>
<script src="./index.js"></script>
```

- Sugar
- Milk
- Eggs

Working with the `innerText()`, `innerHTML()`, `createElement()` and `textContent()` Methods

- `innerHTML` **Property**: This is a property of the `Element` that is used to set or update parts of the HTML markup.

index.html index.js

```
const container = document.getElementById("container");
container.innerHTML = '<ul><li>Cheese</li><li>Tomato</li></ul>';
```

- Cheese
- Tomato

- `createElement` **Method**: This is used to create an HTML element.

```
<div id="container">
  <p>Hello, World!</p>
  <p>I'm learning JavaScript</p>
</div>
<script src="./index.js"></script>
```

Hello, World!

I'm learning JavaScript

Open Sandbo

- `textContent`: This returns the plain text content of an element, including all the text within its descendants.

index.html index.js

```
<div id="container">
  <p>Hello, World!</p>
  <p>I'm learning JavaScript</p>
</div>
<script src="./index.js"></script>
```

Working with the `appendChild()` and `removeChild()` Methods

- `appendChild()` **Method**: This method is used to add a node to the end of the list of children of a specified parent node.

index.html index.js

```
const dessertsList = document.getElementById("desserts");
const listItem = document.createElement("li");

listItem.textContent = "Cookies";
dessertsList.appendChild(listItem);
```

- Cake
- Pie
- Cookies

- `removeChild()` **Method**: This method is used to remove a node from the DOM.

index.html index.js

```
<section id="example-section">
  <h2>Example sub heading</h2>
  <p>first paragraph</p>
  <p>second paragraph</p>
</section>
<script src="./index.js"></script>
```

Example sub heading

first paragraph

Open Sandbo

Work with the `setAttribute()` Method

- **Definition**: This method is used to set the attribute for a given element. If the attribute already exists, then the value is updated. Otherwise, a new attribute is added with a value.

index.html index.js

```
const para = document.getElementById("para");
para.setAttribute("class", "my-class");
```

I am a paragraph

Open Sandbo

Event Object

`addEventListener()` and `removeEventListener()` Methods

- **`addEventListener` Method**: This method is used to listen for events. It takes two arguments: the event you want to listen for and a function that will be called when the event occurs. Some common examples of events would be click events, input events, and change events.

index.html index.js

```
<button id="btn">Click Me</button>
<script src="./index.js"></script>
```

Click Me

Open Sandbox

- **`removeEventListener()` Method**: This method is used to remove an event listener that was previously added to an element using the `addEventListener()` method. This is useful when you want to stop listening for a particular event on an element.

index.html index.js

```
const bodyEl = document.querySelector("body");
const para = document.getElementById("para");
const btn = document.getElementById("btn");

let isBgColorGrey = true;

function toggleBgColor() {
  bodyEl.style.backgroundColor = isBgColorGrey ? "blue" : "grey";
  isBgColorGrey = !isBgColorGrey;
}

btn.addEventListener("click", toggleBgColor);

para.addEventListener("mouseover", () => {
  btn.removeEventListener("click", toggleBgColor);
});
```

- **Inline Event Handlers**: Inline event handlers are special attributes on an HTML element that are used to execute JavaScript code wh an event occurs. In modern JavaScript, inline event handlers are not considered best practice. It is preferred to use the `addEventListener` method instead.

```
<button onclick="alert('Hello World!')">Show alert</button>
```

Show alert

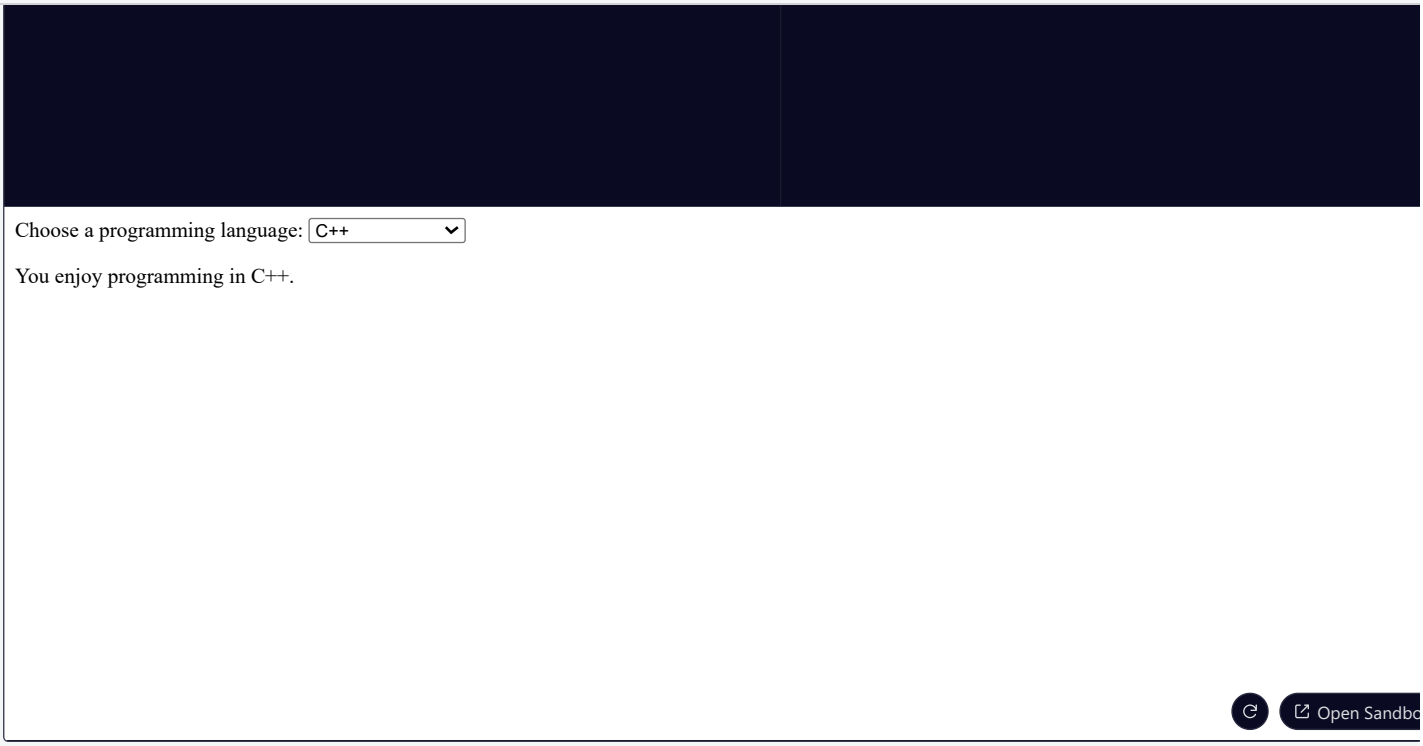
The Change Event

- **Definition**: The change event is a special event which is fired when the user modifies the value of certain input elements. Examples would include when a checkbox or a radio button is ticked. Or when the user makes a selection from something like a date picker or dropdown menu.

index.html index.js

```
const selectEl = document.querySelector(".language");
const result = document.querySelector(".result");

```



Event Bubbling

- **Definition**: Event bubbling, or propagation, refers to how an event "bubbles up" to parent objects when triggered.
- `stopPropagation()` **Method**: This method prevents further propagation for an event.

Event Delegation

- **Definition**: Event delegation is the process of listening to events that have bubbled up to a parent, rather than handling them directly on the element that triggered them.

DOMContentLoaded

- **Definition**: The `DOMContentLoaded` event is fired when everything in the HTML document has been loaded and parsed. If you have external stylesheets, or images, the `DOMContentLoaded` event will not wait for those to be loaded. It will only wait for the HTML to loaded.

Working with `style` and `classList`

- `Element.style` **Property**: This property is a read-only property that represents the inline style of an element. You can use this property to get or set the style of an element.

index.html index.js

```
const paraEl = document.getElementById("para");
paraEl.style.color = "red";
```

Open Sandbo

- `Element.classList` **Property**: This property is a read-only property that can be used to add, remove, or toggle classes on an elem

index.html styles.css index.js

```
<link rel="stylesheet" href="./styles.css"/>
<p id="para" class="blue-background">This paragraph will have classes ad
<div id="menu" class="menu">Menu Content</div>
<button id="toggle-btn">Toggle Menu</button>
<script src="./index.js"></script>
```

This paragraph will have classes added and removed.

Toggle Menu

Open Sandbo

Working with the `setTimeout()` and `setInterval()` Methods

- `setTimeout()` **Method**: This method lets you delay an action for a specified time.

```
setTimeout(() => {
 console.log('This runs after 3 seconds');
}, 3000);
```

```
"This runs after 3 seconds"
```

- **`setInterval()` Method**: This method keeps runs a piece of code repeatedly at a set interval. Since `setInterval()` keeps execut the provided function at the specified interval, you might want to stop it. For this, you have to use the `clearInterval()` method.

```
setInterval(() => {
 console.log('This runs every 2 seconds');
}, 2000);

// Example using clearInterval
const intervalID = setInterval(() => {
 console.log('This will stop after 5 seconds');
}, 1000);

setTimeout(() => {
 clearInterval(intervalID);
}, 5000);
```

```
"This runs every 2 seconds"
"This runs every 2 seconds"
"This runs every 2 seconds"
"This runs every 2 seconds"
"This runs every 2 seconds"
"This runs every 2 seconds"
"This runs every 2 seconds"
"This runs every 2 seconds"
"This runs every 2 seconds"
```

The `requestAnimationFrame()` Method

```
  // Update the animation...
  // for example, move an element, change a style, and more.
  update();
  // Request the next frame
  requestAnimationFrame(animate);
 }
```

Web Animations API

- **Definition**: The Web Animations API lets you create and control animations directly inside JavaScript.

index.html styles.css index.js

```
const square = document.querySelector('#square');

const animation = square.animate(
 [{ transform: 'translateX(0px)' }, { transform: 'translateX(100px)' }],
 {
   duration: 2000, // makes the animation last 2 seconds
   iterations: Infinity, // loops indefinitely
   direction: 'alternate', // moves back and forth
   easing: 'ease-in-out', // smooth easing
 }
);
```

Open Sandbo

The Canvas API

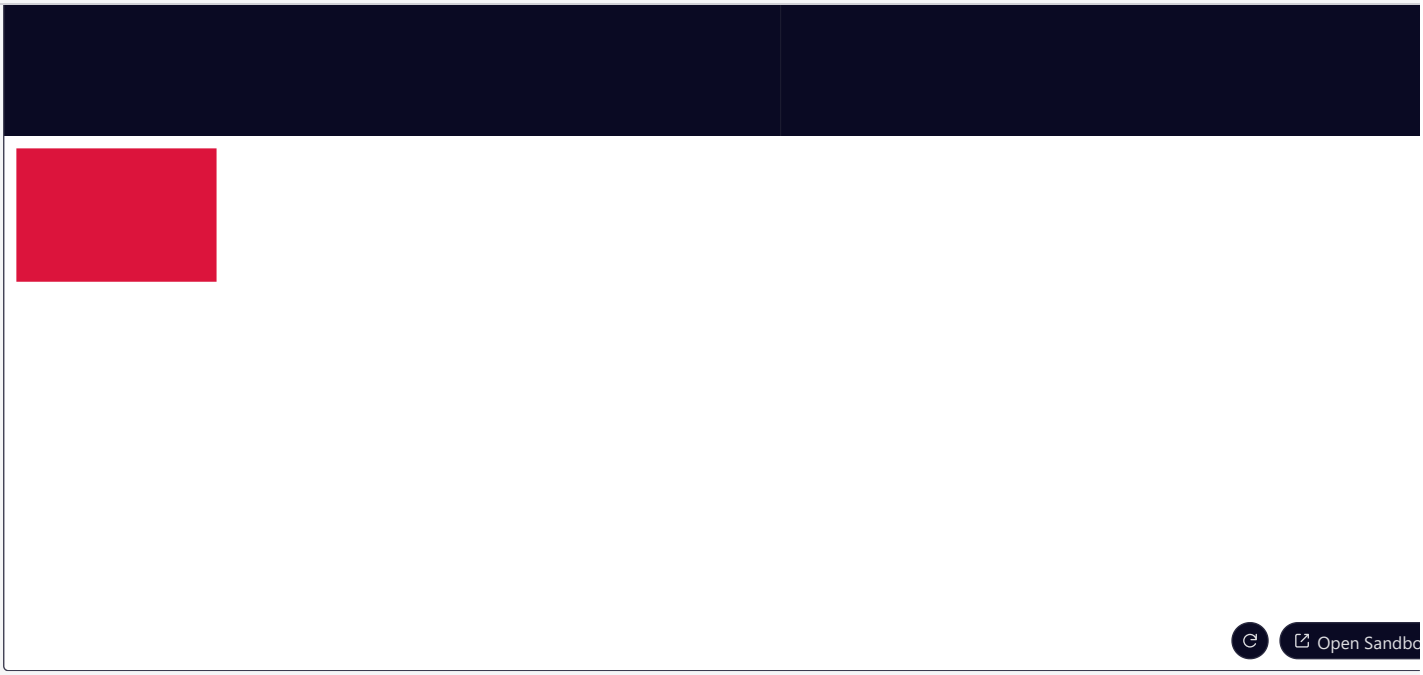
- **Definition**: The Canvas API is a powerful tool that lets you manipulate graphics right inside your JavaScript file. To work with the Canvas API, you first need to provide a `canvas` element in HTML. This element acts as a drawing surface you can manipulate with t instance methods and properties of the interfaces in the Canvas API. This API has interfaces like `HTMLCanvasElement`, `CanvasRenderingContext2D`, `CanvasGradient`, `CanvasPattern`, and `TextMetrics` which contain methods and properties yo can use to create graphics in your JavaScript file.

index.html index.js

```
const canvas = document.getElementById('my-canvas');

// Access the drawing context of the canvas.
// "2d" allows you to draw in two dimensions
const ctx = canvas.getContext('2d');

```



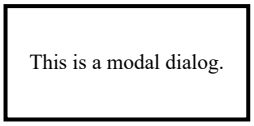
Opening and Closing Dialogs and Modals with JavaScript

- **Modal and Dialog Definitions**: Dialogs let you display important information or actions to users. With the HTML built-in dialog element, you can easily create these dialogs (both modal and non-modal dialogs) in your web apps. A modal dialog is a type of dialog that forces the user to interact with it before they can access the rest of the application or webpage. In contrast, a non-modal dialog allows the user to continue interacting with other parts of the page or application even when the dialog is open. It doesn't prevent access to the rest of the content.
- `showModal()` **Method**: This method is used to open a modal.

index.html index.js

```
const dialog = document.getElementById('my-modal');
const openButton = document.getElementById('open-modal');

openButton.addEventListener('click', () => {
  dialog.showModal();
});
```



Open Sandbo

- `close()` **Method**: This method is used to close the modal.

index.html index.js

```
<dialog id="my-modal">
   <p>This is a modal dialog.</p>
   <button id="close-modal">Close Modal</button>
</dialog>
<button id="open-modal">Open Modal Dialog</button>
<script src="./index.js"></script>
```

Open Modal Dialog

Open Sandbo

Assignment

- [ ] Review the DOM Manipulation and Click Events with JavaScript topics and concepts.

Submit

Ask for Help